

```
In [26]: import pandas as pd      #Step 1:Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

```
In [9]: df=pd.read_csv('C:/Users/hp/Desktop/loan_eligibility.csv')  #Step 2:Load Datas
print(df.head())
```

	ApplicantID	Gender	Married	Dependents	Education	Self_Employed	\
0	1	Male	Yes	0	Graduate	Yes	
1	2	Female	Yes	3+	Not Graduate	No	
2	3	Male	Yes	3+	Not Graduate	No	
3	4	Male	No	1	Graduate	No	
4	5	Male	No	2	Graduate	Yes	

	ApplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	\
0	3970	251	360	1.0	
1	10389	287	360	1.0	
2	19586	197	360	1.0	
3	17374	230	180	1.0	
4	7788	270	360	1.0	

	Property_Area	Loan_Status
0	Urban	Y
1	Semiurban	Y
2	Semiurban	N
3	Rural	N
4	Semiurban	Y

```
In [10]: print(df.info())      #Step 3:Check Info and Nulls
print(df.isnull().sum())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   ApplicantID           300 non-null    int64  
 1   Gender                 300 non-null    object  
 2   Married                300 non-null    object  
 3   Dependents             300 non-null    object  
 4   Education              300 non-null    object  
 5   Self_Employed          300 non-null    object  
 6   ApplicantIncome        300 non-null    int64  
 7   LoanAmount             300 non-null    int64  
 8   Loan_Amount_Term       300 non-null    int64  
 9   Credit_History         300 non-null    float64 
10   Property_Area          300 non-null    object  
11   Loan_Status            300 non-null    object  
dtypes: float64(1), int64(4), object(7)
memory usage: 28.3+ KB
None
ApplicantID      0
Gender            0
Married          0
Dependents       0
Education        0
Self_Employed    0
ApplicantIncome  0
LoanAmount       0
Loan_Amount_Term 0
Credit_History   0
Property_Area    0
Loan_Status      0
dtype: int64

```

```

In [25]: le = LabelEncoder()           #Step 4:Encode Categorical Variables

categorical_cols = ['Gender', 'Married', 'Dependents', 'Education',
                    'Self_Employed', 'Property_Area', 'Loan_Status']

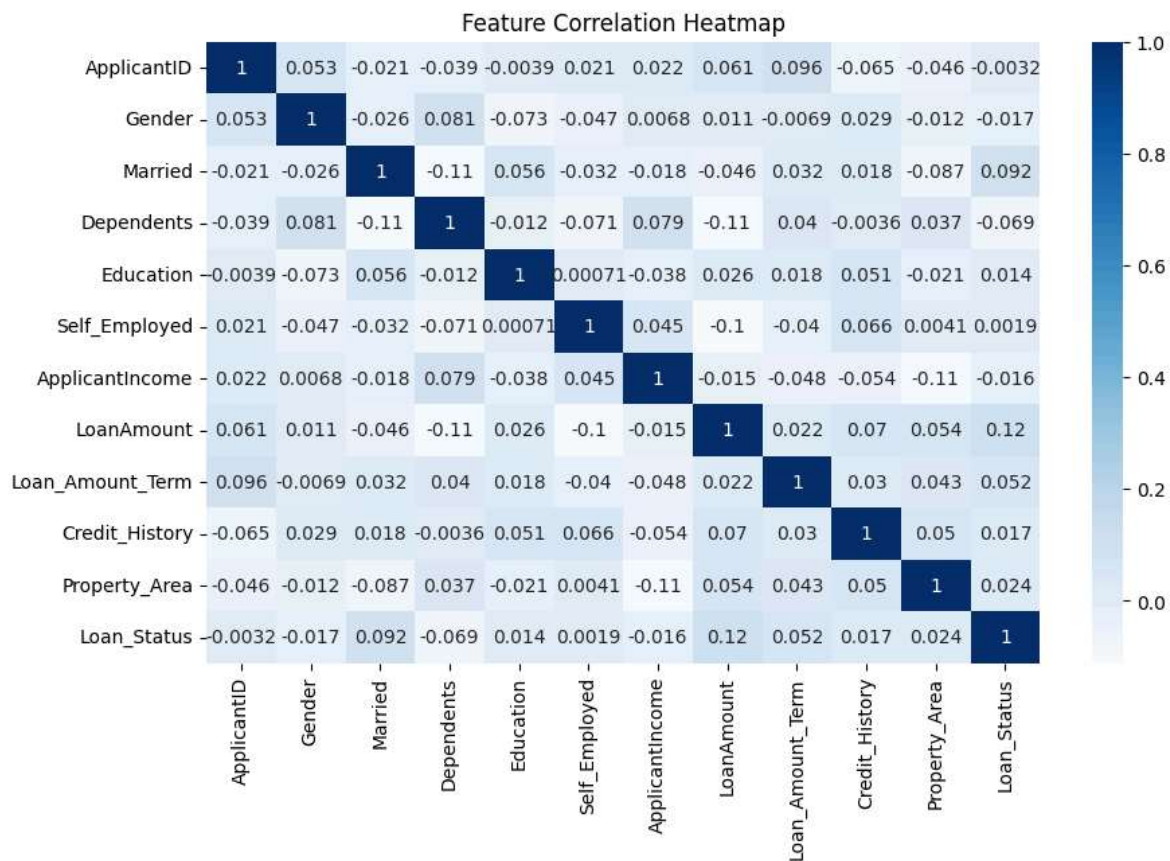
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

```

```

In [27]: plt.figure(figsize=(10,6))    #Step 5:Correlation Heatmap
sns.heatmap(df.corr(), annot=True, cmap='Blues')
plt.title("Feature Correlation Heatmap")
plt.show()

```



```
In [28]: X = df.drop(['ApplicantID', 'Loan_Status'], axis=1) #Step 6:Feature and Target
         y = df['Loan_Status']
```

```
In [29]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
In [30]: model = RandomForestClassifier() #Step 8:Train the Model
         model.fit(X_train, y_train)
```

```
Out[30]: ▼ RandomForestClassifier ⓘ ?
```

```
RandomForestClassifier()
```

```
In [31]: y_pred = model.predict(X_test) #Step 9:Evaluate the Model
         print(confusion_matrix(y_test, y_pred))
         print(classification_report(y_test, y_pred))
```

```
[[ 0 25]
 [ 1 34]]
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	25
1	0.58	0.97	0.72	35
accuracy			0.57	60
macro avg	0.29	0.49	0.36	60
weighted avg	0.34	0.57	0.42	60

```
In [33]: #Example input: Male, Married, 2 dependents, Graduate, Self-Employed, Income = 5
         new_data = pd.DataFrame({
             'Gender': [1],
             'Married': [1],
```

```
'Dependents': [2],  
'Education': [1],  
'Self_Employed': [1],  
'ApplicantIncome': [5000],  
'LoanAmount': [200],  
'Loan_Amount_Term': [360],  
'Credit_History': [1.0],  
'Property_Area': [2]  
})  
  
prediction = model.predict(new_data)  
print("Loan Eligibility Prediction (1=Eligible, 0=Not Eligible):", prediction[0])
```

Loan Eligibility Prediction (1=Eligible, 0=Not Eligible): 1

In []: